



АЛГОРИТМЫ И ПРОИЗВОДИТЕЛЬНОСТЬ

ЛЕКЦИЯ 2. Абстрактные типы данных и вычислительная сложность их реализаций

Пестунов Андрей Игоревич

Контакты для переписки:
andrey.pestunov@yandex.ru
<https://vk.ru/id572960176>

АБСТРАКТНЫЕ ТИПЫ ДАННЫХ (АДТ)

Абстрактный тип данных – это тип данных, определяемый **интерфейсом** его взаимодействия с пользователем и содержащим доступные к выполнению операции без конкретизации их реализации

Цель лекции - рассмотреть, как на основе массива реализуются абстрактные типы данных (АТД) **стек**, **очередь**, **список** и **множество**. Уяснить различие между интерфейсом АТД и его реализацией. Оценить вычислительную **сложность** реализации этих операций

- АТД определяет **только сигнатуру** операций, поддерживаемых типом
- **Реализации** могут быть **различными**
- АТД определяет **интерфейс взаимодействия** с дискретными и другими структурами данных
- Другие АТД: **ассоциативный** массив, очередь по **приоритетам**

“ПЕРЕМЕННАЯ” И “МАССИВ” КАК БАЗОВЫЕ АБСТРАКТНЫЕ ТИПЫ ДАННЫХ

Переменная (англ. - variable).

Операции:

store(x) – **присвоить** значение переменной;

fetch() – **извлечь** значение переменной (обратиться к переменной).

Массив (англ. - array).

Операции:

store(i, x) – **присвоить** значение элемента массива с заданным **индексом**;

fetch(i) – **извлечь** значение элемента с указанным **индексом**.

Присвоение обозначается традиционно “:=” или “=”, а извлечение значения - это просто указание имени переменной или имени массива с индексом элемента в выражении: “**elements[i]**”

МАССИВ – СТАТИЧЕСКИЙ ТИП ДАННЫХ

Интерфейс массива **не имеет операций**, позволяющих менять размер, добавлять или удалять элементы

Для варьирования размера нужны **дополнительные манипуляции**

- **Изменение** размера;
- **Добавление** элемента;
- **Вставка** элемента в указанное место;
- **Удаление** элемента;
- Проверка на **пустоту**;
- Проверка на **заполненность**;
- **Быстрый** поиск;

```
int mas[100];
int n;
cout << "Введите размер массива:\n";
cin >> n;

for (int i=0; i<n; i++) {
    mas[i] = rand()%100;
}

for (int i=0; i<n; i++) {
    cout << mas[i] << " ";
}
```

ДИНАМИЧЕСКИЕ И СТАТИЧЕСКИЕ СТРУКТУРЫ

Статические структуры – это структуры, интерфейс которых не содержит операции, позволяющие изменять их в процессе выполнения программы данных. Их размер и структура задается перед запуском программы. Изменять можно только значения

Динамические структуры – это структуры, интерфейс которых, содержит операции которые могут изменяют размер и структуру в процессе выполнения программы

Примеры **динамических** типов данных: стек, очередь, множество, очередь по приоритетам

Принципы реализации динамических операций:

- **Имитация** с помощью **пометки** элементов (использование «**флажков**»)
- **Имитация** с помощью **счетчика** фактического количества элементов
- **Динамическое распределение** памяти (**односвязные** и **двусвязные** списки)

СТЕК КАК АБСТРАКТНЫЙ ТИП ДАННЫХ

«Стек» (переводится «стопка»). Этот АТД реализует стратегию «**Последним пришел - первым вышел**» (last in - first out). Сокращенно - LIFO. Этим обуславливается наличие у него двух основных операций: **добавление** (в конец) и **удаление последнего** элемента. Извлечь значение можно только у последнего элемента стека.

Англ. - Stack.

Операции:

push(x) – вставить элемент (в конец стека);

pop() – удалить последний (самый “новый”) элемент;

last() – получить значение последнего элемента;
другое название - top();

isEmpty() – проверка на пустоту (true – пустой;
false – хотя бы 1 элемент).



СТЕК КАК СТРУКТУРА ДАННЫХ

Абстрактный тип данных – программный **интерфейс**, определяющий **сигнатуру** операций над некоторым объектом (структурой данных)

Структура данных – логическая **концепция**, определяющая **принципы** реализации АД и их вычислительную сложность

Программная реализация – **код** на конкретном языке программирования, **воплощающий** принципы, определяемые структурой данных

[illegible]

ОЧЕРЕДЬ КАК АБСТРАКТНЫЙ ТИП ДАННЫХ

Поведение очереди вполне соответствует ее названию. Добавить элемент можно только в конец очереди (встать последним в очередь), а удалить элемент можно только из начала (обслуживается первый человек в очереди, вставший раньше других). АДТ “очередь” реализует стратегию **“Первым пришел - первым вышел”** (first in - first out). Сокращенно - FIFO.

Англ. - Queue.

Операции:

enqueue(x) – вставить элемент на последнее место (в конец очереди);

dequeue() – удалить первый (самый “старый”) элемент;

first() – получить значение первого элемента; другое название - peek();

isEmpty() – проверка на пустоту (true – пустая; false – хотя бы 1 элемент).

ОЧЕРЕДЬ КАК СТРУКТУРА ДАННЫХ

								P	
			M				K	K	P
		R	R	M		Y	Y	Y	K
	T	T	T	R	M	M	M	M	Y
A	A	A	A	T	R	R	R	R	M

СПИСОК КАК АБСТРАКТНЫЙ ТИП ДАННЫХ

Список дает доступ не только к первому/последнему элементу (как стек и очередь), но и позволяет **обращаться к любой позиции по индексу**.

Англ. - List.

Операции:

insert(i, x) – вставить элемент в указанное место

remove(i) – удалить элемент с заданным индексом

get(i) – поиск – значение на позиции с указанным индексом

indexOf(x) – проверка наличия элемента в списке

isEmpty() – проверка на пустоту (true – пустой; false – хотя бы 1 элемент)

Вычислительная сложность выполнения этих операций зависит от реализации:

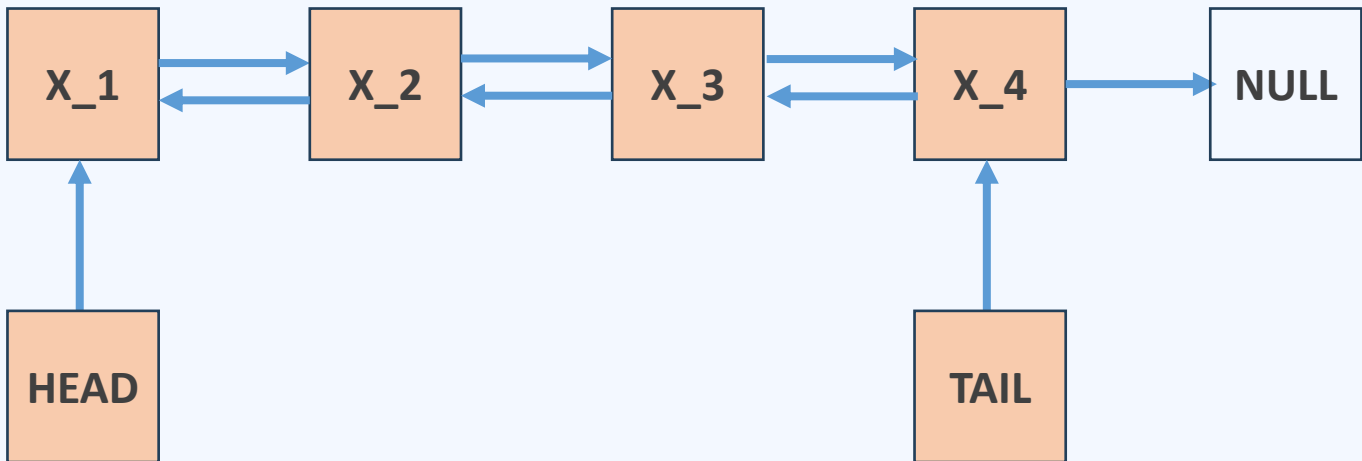
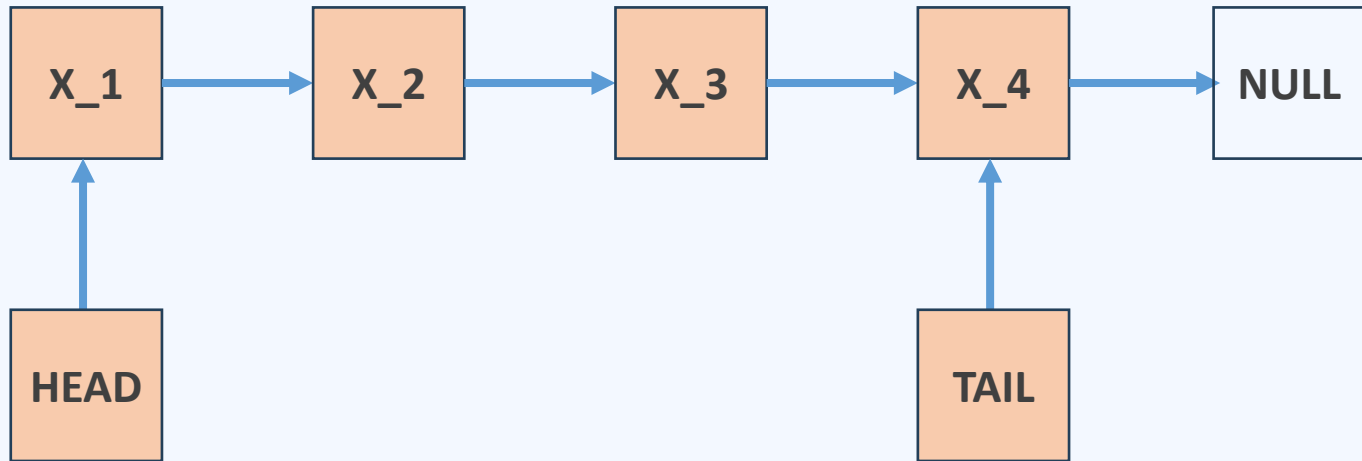
- На основе **массива**
- На основе **односвязного** списка (динамическое распределение памяти)
- На основе **двусвязного** списка (динамическое распределение памяти)

СПИСОК КАК СТРУКТУРА ДАННЫХ

Вставка/удаление в любом месте

								T	
			R				T	Y	T
		T	T	R		T	Y	M	Y
	T	R	A	T	T	Y	M	K	M
A	A	A	M	M	M	M	K	P	P

ОДНОСВЯЗНЫЕ И ДВУСВЯЗНЫЕ СПИСКИ – ДИНАМИЧЕСКИЕ СТРУКТУРЫ ДАННЫХ



ИДЕЯ РЕАЛИЗАЦИЯ СТЕКА

- Воспользуемся идеей работы с массивом заранее неизвестного размера.
 - Создадим массив, размер которого равен предполагаемой вместимости стека.
 - Объявим **дополнительную переменную**, которая будет хранить текущий размер массива и указывать на последний элемент.
 - Будем **увеличивать** эту переменную на 1 при добавлении элемента и **уменьшать** при удалении.

РЕАЛИЗАЦИЯ СТЕКА

```
const int max_size=10;  
string contents[max_size];  
int n=0;
```

```
void print() {  
    for (int i=0; i<n; i++) {  
        cout << contents[i] << " ";  
    }  
    cout << endl;  
}
```

СТЕК: РЕАЛИЗАЦИЯ ДОБАВЛЕНИЯ И УДАЛЕНИЯ

```
void push(string x) {  
    if (n==max_size) {  
        cout << "Stack is full\n";  
    } else {  
        contents[n] = x;  
        n++;  
    }  
}
```

```
void pop() {  
    if (n==0) {  
        cout << "Stack is empty\n";  
    } else {  
        n--;  
    }  
}
```

ПРИМЕР ИСПОЛЬЗОВАНИЯ

```
for (;;) {
    cout << "1 - Print\n2 - Push\n3 - Pop\n";
    cout << "0 - Exit\nYour choice - ";
    int choice; cin >> choice;
    string newstr;
    switch(choice) {
        case 1 : print(); break;
        case 2 :
            cout << "New string: ";
            cin >> newstr;
            push(newstr);
            print();
            break;
        case 3 : pop(); print(); break;
        case 0 : exit(EXIT_SUCCESS);
        default : cout << "Error\n";
    }
    system("pause"); system("cls");
}
```

ЭЛЕМЕНТЫ МАССИВА ХРАНЯТСЯ В ПАМЯТИ ПОСЛЕДОВАТЕЛЬНО

//Способ 1.

```
int mas[] = {10,20,30,40,50};
```

```
printf("%9s%9s%9s\n", "И", "З", "А");
```

```
for (int i=0; i<5; i++) {
```

```
    printf("%9d%9d%9d\n", i, mas[i], &mas[i]);
```

```
}
```

//Способ 2.

```
int mas[] = {10,20,30,40,50};
```

```
printf("%9s%9s%9s\n", "Индекс", "Значение", "Адрес");
```

```
for (int i=0; i<5; i++) {
```

```
    printf("%9d%9d%9d\n", i, *(mas+i), mas+i);
```

```
}
```

C:\ X:\ucosz\bin\Debug\ucosz.exe

Индекс	Значение	Адрес
0	10	2293528
1	20	2293532
2	30	2293536
3	40	2293540
4	50	2293544

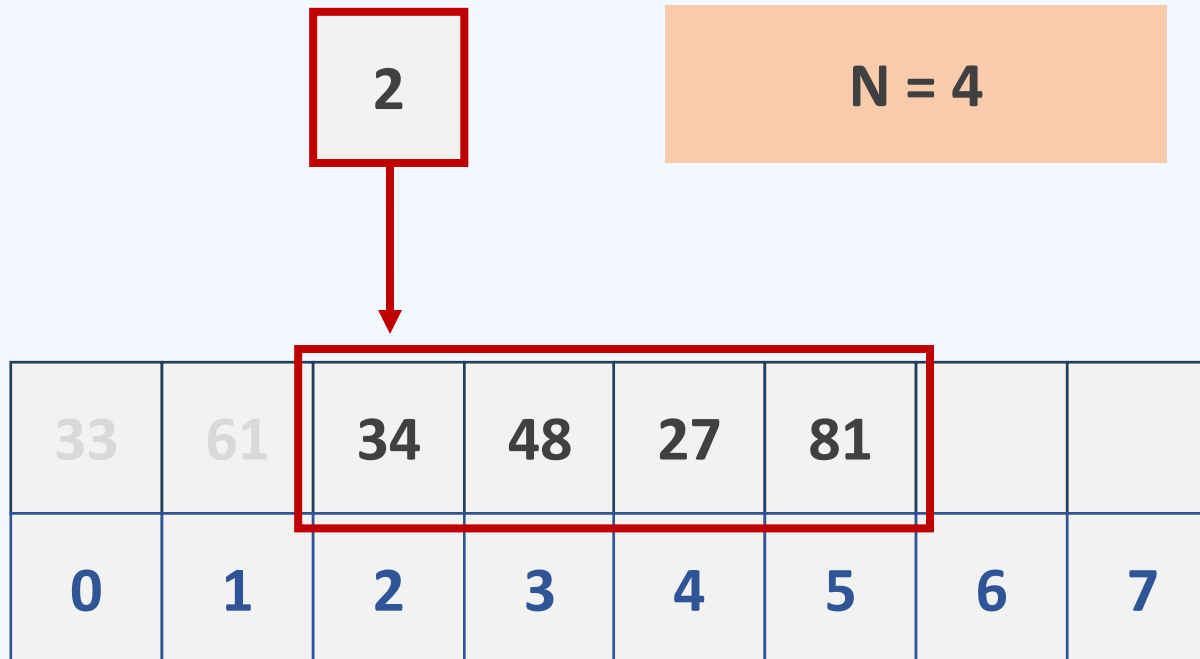
РЕАЛИЗАЦИЯ ОЧЕРЕДИ

- Реализация очереди аналогична стеку, за исключением операции удаления: следует **сдвинуть** следующие за первым удаляемым элементом на его место.

```
void remove() {  
    if (n==0) {  
        cout << "Queue is empty\n";  
    } else {  
        //Сдвиг элементов  
        for (int i=0; i<n-1; i++) {  
            contents[i] = contents[i+1];  
        }  
        n--;  
    }  
}
```

ОПТИМИЗАЦИЯ РЕАЛИЗАЦИИ ОЧЕРЕДИ

- **Удаление** первого элемента очереди – **трудоемкая** процедура при реализации на основе массива, требуется сдвиг элементов
- Оптимизация может состоять в том, чтобы заменить сдвиг перемещением **переменной-указателя**
- Это возможно при наличии **резервной** памяти



РЕАЛИЗАЦИЯ АД «СПИСОК»

- При вставке элемента на некоторую позицию нужно **отодвинуть элементы, расположенные справа** от этой позиции, освободив место новому элементу.
- При удалении нужно **сдвинуть влево элементы, расположенные правее** удалённого, чтобы закрыть «дыру».

ВСТАВКА ЭЛЕМЕНТА В СПИСОК

```
void insert(double x, int pos) {
    if (n==max_size) {
        cout << "List is full\n";
    } else if (!(0<=pos && pos<=n)) {
        cout << "Incorrect index\n";
    } else {
        //СДВИГ элементов вправо
        for (int i=n-1; i>=pos; i--) {
            contents[i+1] = contents[i];
        }
        contents[pos] = x;
        n++;
    }
}
```

УДАЛЕНИЕ ИЗ СПИСКА

```
int find(double x) {  
    for (int i=0; i<n; i++) {  
        if (contents[i] == x) {  
            return i;  
        }  
    }  
    return -1;  
}
```

```
void remove(double x) {  
    int pos = find(x);  
    if (pos==-1) {  
        cout << "No such element\n";  
    } else {  
        //СДВИГ ЭЛЕМЕНТОВ ВЛЕВО  
        for (int i=pos; i<n-1; i++) {  
            contents[i] = contents[i+1];  
        }  
        n--;  
    }  
}
```

ВЫВОД СПИСКА АНАЛОГИЧЕН СТЕКУ И ОЧЕРЕДИ

```
void print() {  
    //Проверка на пустоту  
    if (n==0) {  
        cout << "List is empty\n";  
    }  
    for (int i=0; i<n; i++) {  
        cout << contents[i] << " ";  
    }  
    cout << endl;  
}
```

МНОЖЕСТВО

Каждый элемент, содержащийся во множестве, наличествует **ровно в одном экземпляре**. Множество - это неупорядоченный набор элементов. Порядок добавления или порядковый номер элемента значения не имеет. Важно лишь наличие элемента во множестве.

Англ. - Set.

Операции:

add(x) – добавить элемент (не выполняется, если уже есть во множестве);
remove(x) – удалить элемент с указанным значением;
contains(x) – проверить наличие элемента во множестве;
isEmpty() – проверка на пустоту (true – пустое; false – хотя бы 1 элемент).

ИМИТАЦИЯ ДОБАВЛЕНИЯ И УДАЛЕНИЯ



ИСПОЛЬЗОВАНИЕ ПОМЕТКИ ЭЛЕМЕНТОВ ДЛЯ ИМИТАЦИИ ДОБАВЛЕНИЯ/УДАЛЕНИЯ

Элементы	Петров	Цветков	Сидоров	Кротов	Сергеев	Гусев
Флажок	Есть	Нет	Есть	Есть	Нет	Есть

Вывести все элементы

Цикл $i=0..5$

Если (Флажок[i] == Есть), то

Вывести(Элементы[i]);

Удаление

Флажок[2] := Нет.

РЕАЛИЗАЦИЯ АДТ «МНОЖЕСТВО»

```
const int size=10;
int numbers[size];
bool exists[size];

void init() {
    for (int i=0; i<size; i++) {
        exists[i]=false;
    }
}
```

```
void print() {
    for (int i=0; i<size; i++) {
        if (exists[i]) {
            cout << numbers[i] << " ";
        }
    }
    cout << endl;
}
```

РЕАЛИЗАЦИЯ ДОБАВЛЕНИЯ ВО МНОЖЕСТВО

```
void add(int x) {  
    int pos = getEmptyPosition();  
    if (pos != -1) {  
        numbers[pos]=x;  
        exists[pos]=true;  
    } else {  
        cout << "No place\n";  
    }  
}
```

```
int getEmptyPosition() {  
    for (int i=0; i<size; i++) {  
        if (!exists[i]) {  
            return i;  
        }  
    }  
    return -1;  
}
```

ПОИСК И УДАЛЕНИЕ ВО МНОЖЕСТВЕ

```
bool contains(int x) {  
    for (int i=0; i<size; i++) {  
        if (exists[i] && numbers[i]==x) {  
            return true;  
        }  
    }  
    return false;  
}
```

```
void remove(int x) {  
    int pos = find(x);  
    if (pos != -1) {  
        exists[pos]=false;  
    } else {  
        cout << "No such element\n";  
    }  
}
```

ОБЕРТКА ЭЛЕМЕНТА И ФЛАЖКА ВМЕСТЕ

```
struct Element {  
    int item;  
    bool exists;  
};  
  
const int size=10;  
Element numbers[size];  
  
void init() {  
    for (int i=0; i<size; i++) {  
        numbers[i].exists = false;  
    }  
}
```

```
void print() {  
    for (int i=0; i<size; i++) {  
        if (numbers[i].exists) {  
            cout << numbers[i].item << " ";  
        }  
    }  
    cout << endl;  
}
```

ДОБАВЛЕНИЕ ПРИ ИСПОЛЬЗОВАНИИ ОБЕРТКИ

```
int getEmptyPosition() {
    for (int i=0; i<size; i++) {
        if (!numbers[i].exists) {
            return i;
        }
    }
    return -1;
}

void add(int x) {
    int pos = getEmptyPosition();
    if (pos != -1) {
        numbers[pos].item=x;
        numbers[pos].exists=true;
    } else {
        cout << "No place\n";
    }
}
```

Пример использования операций

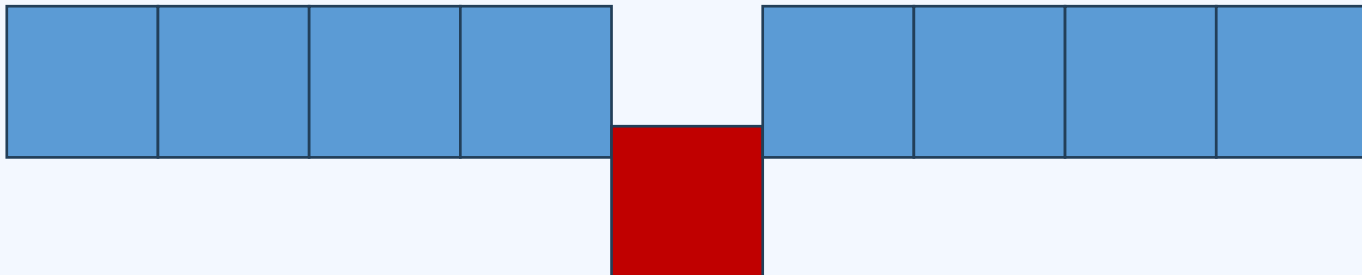
```
for (;;) {
    cout << "List: "; print();
    cout << "1 - Add\n2 - Remove\n";
    cout << "0 - Exit\nYour choice - ";
    int choice; cin >> choice;
    double toInsert, toRemove;
    int pos;
    switch(choice) {
        case 1 :
            cout << "New number: ";
            cin >> toInsert;
            cout << "Position: ";
            cin >> pos;
            insert(toInsert, pos);
            break;
        case 2 :
            cout << "To remove: ";
            cin >> toRemove;
            remove(toRemove);
            break;
        case 0 : exit(EXIT_SUCCESS);
        default : cout << "Error\n";
    }
    system("pause"); system("cls");
}
```

О СЛОЖНОСТИ ОПЕРАЦИЙ

- Удалить элемент из **стека** – это 1 операция
- Удалить элемент из **очереди** – это еще и сдвиг.
- Для этого выполняется порядка n операция
 - **Список** тоже использует сдвиг

РЕАЛИЗАЦИЯ МНОЖЕСТВА НА ОСНОВЕ УПОРЯДОЧЕННОГО МАССИВА

- **Поиск** в упорядоченном массиве осуществляется **быстро** – за логарифмическое время
- Необходимо уделять внимание поддержанию упорядоченности при осуществлении **вставки** и **удалении**
- При вставке – **сдвигать** элементы, чтобы освободить место под вставляемый элемент
- При удалении – **сдвигать** элементы, чтобы закрыть пустое место, где находился удаляемый элемент
- Вычислительная сложность сдвигов – **линейная**, $O(N)$
- Реализация множества на основе упорядоченного массива **выгодна** при **большом** количестве поисковых запросов и **малом** количестве вставок/удалений



ВЫЧИСЛИТЕЛЬНАЯ СЛОЖНОСТЬ ДВУХ РЕАЛИЗАЦИЙ МНОЖЕСТВА

	На основе неупорядоченного массива	На основе упорядоченного массива
Вставка	$O(1)$	$O(N)$
Удаление	$O(N)$	$O(N)$
Поиск	$O(N)$	$O(\log_2(N))$

Удаление занимает **линейное** время в **обоих** случаях
Идея для эксперимента – варьировать **доли** запросов
каждого вида и определить при каких моделях входных
данных **выгодна** одна из этих двух реализаций

ДИНАМИЧЕСКОЕ РАСПРЕДЕЛЕНИЕ ПАМЯТИ И ОПЕРАТОР NEW

```
int* ptr;  
int size;  
printf("Введите размер: ")
```

```
//!!!!!!  
scanf("%d", &size);  
ptr = new int[size];  
//!!!!!!
```

```
for (int i=0; i<size; i++)  
*(ptr+i) = rand();  
}
```

```
for (int i=0; i<size; i++)  
printf("%-7d\\n", ptr[i]);  
}
```

Динамическое распределение –
управление памятью **в процессе**
исполнения программы

Статическое распределение – выделение
памяти **перед запуском** программы и
сохранение структуры выделенной
памяти в процессе исполнения

```
Racer1 racer1 = new Racer1();  
Racer2 racer2 = new Racer2();
```

```
Racer3 racer3 = new Racer3();  
Thread threadForRacer3 = new Thread(racer3);
```

```
racer1.start();  
racer2.start();  
threadForRacer3.start();
```

```
}  
}
```

ЭЛЕМЕНТЫ СТРОКИ (СИМВОЛЬНОГО МАССИВА) ХРАНЯТСЯ В ПАМЯТИ ПОСЛЕДОВАТЕЛЬНО

```
//Напишите программу, которая выводит полученную  
//с клавиатуры строку наоборот. Работу со строкой  
//производите с помощью указателей.
```

```
int main() {  
    char* str = new char[100];  
    gets(str);  
  
    int l = strlen(str);  
    for (int i=0; i<l; i++) {  
        cout << *(str+l-i-1);  
    }  
    cout << endl;  
    return 0;  
}
```

МАНИПУЛЯЦИЯ ЗНАЧЕНИЯМИ ЭЛЕМЕНТОВ МАССИВА ЧЕРЕЗ УКАЗАТЕЛИ

```
//Напишите программу, которая сортирует массив  
//по возрастанию методом прямого выбора  
//с помощью указателей.
```

```
for (int i=0; i<size-1; i++) {  
    int j_min = *(mas+i);  
    for (int j=i+1; j<size; j++) {  
        if (*(mas+j_min) > *(mas+j)) {  
            j_min = j;  
        }  
    }  
    int temp = *(mas+i);  
    *(mas+i) = *(mas+j_min);  
    *(mas+j_min) = temp;  
}
```

```
//Написать функцию, которая принимает указатели  
//на две вещественных переменных и меняет  
//их значения местами.
```

```
void swap(float* x, float* y) {  
    int temp = *x;  
    *x = *y;  
    *y = temp;  
}
```

```
int main() {  
    float var1 = 10, var2 = 20;  
    cout << "var1=" << var1 << "; var2=" << var2 << endl;  
    swap(&var1, &var2);  
    cout << "var1=" << var1 << "; var2=" << var2 << endl;  
    return 0;  
}
```

```
//Написать функцию, которая принимает два аргумента:  
//указатель на целочисленный массив и размер этого массива.  
//функция должна вернуть количество четных элементов массива.  
//Работа с массивом внутри функции должна быть  
//реализована через указатели.
```

```
int n_evens(int* mas, int size) {  
    int result=0;  
    for (int i=0; i<size; i++) {  
        if ( (*(mas+i))%2 == 0) {  
            result++;  
        }  
    }  
    return result;  
}
```

ОБЪЯВЛЕНИЕ УЗЛА ДЛЯ РЕАЛИЗАЦИИ СТЕКА НА ОСНОВЕ ДИНАМИЧЕСКОГО РАСПРЕДЕЛЕНИЯ ПАМЯТИ

```
//Структура, описывающая элемент стека
struct StackElement {

    //Содержимое стека
    int item;

    //Указатель на расположенный ниже элемент
    StackElement* down;
};
```

Инициализация стека

```
//Указатель на верхний элемент стека
StackElement* top;

//Инициализация стека
void initStack() {
    top = NULL;
}
```

Добавление элемента в стек

```
//Добавление нового элемента в стек
void push(int newItem) {

    //Выделяем память под новый элемент
    StackElement* newElement = new StackElement();

    //Задаем содержимое нового элемента
    newElement->item = newItem;

    //Помещаем новый элемент в конец стека
    newElement->down = top;

    //Сообщаем, что теперь последний элемент -
    //добавленный элемент
    top = newElement;
}
```

Проверка стека на пустоту

```
//Проверка стека на пустоту
bool isEmpty() {
    //Стек пуст, если верхнего элемента нет
    return top == NULL;
}
```

Возврат содержимого верхнего элемента

```
//Возврат содержимого верхнего элемента
int last() {
    //Возвращаем содержимое верхнего элемента
    return top->item;
}
```

Удаление верхнего элемента

```
//Удаление верхнего элемента
void pop() {
    //Если стек пуст, то удалить нельзя
    if (isEmpty()) {
        cout << "The stack is empty." << endl;
    } else {
        //Запоминаем последний (верхний) элемент
        StackElement* toRemove = top;

        //Теперь последний элемент - это предпоследний
        top = top->down;

        //Удаляем бывший верхний элемент
        delete toRemove;
    }
}
```

Вывод стека на экран

```
//Отладочно-проверочная функция
void printStack() {

    //Если стек пуст - то сообщаем об этом
    if (isEmpty()) {
        cout << "The stack is empty." << endl;
    } else {
        //Начинаем с верхнего элемента
        StackElement* temp = top;
        do {
            //Выводим содержимое очередного элемента
            cout << temp->item << " ";
            //Спускаемся вниз
            temp = temp->down;
        }while (temp != NULL);
        cout << endl;
    }
}
```

Тестирование программы

```
int _tmain(int argc, _TCHAR* argv[])
{
    initStack(); printStack();
    push(10);    printStack();
    push(20);    printStack();
    push(30);    printStack();
    push(25);    printStack();
    pop();       printStack();
    pop();       printStack();
    pop();       printStack();
    pop();
    return 0;
}
```

```
The stack is empty.
10
20 10
30 20 10
25 30 20 10
30 20 10
20 10
10
The stack is empty.
```

РЕАЛИЗАЦИЯ УЗЛА ОЧЕРЕДИ

```
//Структура, описывающая элемент очереди
struct QueueElement {

    //Содержимое очереди
    int item;

    //Указатель на расположенный выше элемент
    QueueElement* next;
};
```

Инициализация очереди

```
//Указатель на голову очереди
QueueElement* head;
//Указатель на хвост очереди
QueueElement* tail;

//Инициализация очереди
void initQueue() {
    head = NULL;
    tail = NULL;
}
```

Проверка на пустоту

```
//Проверка очереди на пустоту
bool isEmpty() {
    //Очередь пуста, если головы нет
    return head == NULL;
}
```

Добавление элемента

```
//Добавление нового элемента в очередь
void add(int newItem) {
    //Выделяем память под новый элемент
    QueueElement* newElement = new QueueElement();
    //Задаем содержимое нового элемента
    newElement->item = newItem;

    if (isEmpty()) {
        head = newElement;
    } else {
        //Помещаем новый элемент в конец очереди
        tail->next = newElement;
    }
    //Сообщаем, что теперь последний элемент -
    //добавленный элемент
    tail = newElement;
}
```

Возврат первого элемента

```
//Возврат содержимого головного элемента
int first() {

    //Возвращаем содержимое головного элемента
    return head->item;
}
```

Удаление «головы»

```
//Удаление головного элемента
void remove() {
    //Если очередь пуста, то удалить нельзя
    if (isEmpty()) {
        cout << " The queue is empty." << endl;
    } else {
        //Запоминаем головной элемент
        QueueElement* toRemove = head;

        //Теперь головной элемент - это второй
        head = head->next;

        //Удаляем бывший головной элемент
        delete toRemove;
    }
}
```

Вывод очереди

```
//Отладочно-проверочная функция
void printQueue() {

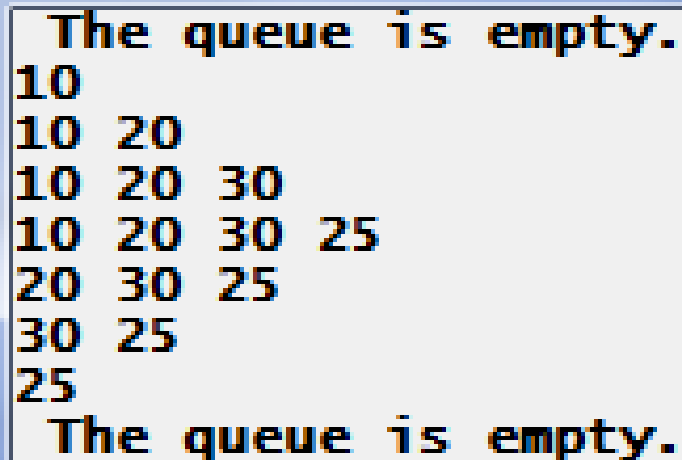
    //Если очередь пуста - то сообщаем об этом
    if (isEmpty()) {
        cout << " The queue is empty." << endl;
    } else {
        //Начинаем с головного элемента
        QueueElement* temp = head;
        do {
            //Выводим содержимое очередного элемента
            cout << temp->item << " ";
            //Переходим к следующему
            temp = temp->next;
        }while (temp != NULL);
        cout << endl;
    }
}
```

Тестирование программы

```
int _tmain(int argc, _TCHAR* argv[])
{

    initQueue(); printQueue();
    add(10);      printQueue();
    add(20);      printQueue();
    add(30);      printQueue();
    add(25);      printQueue();
    remove();     printQueue();
    remove();     printQueue();
    remove();     printQueue();
    remove();     printQueue();

    return 0;
}
```



```
The queue is empty.
10
10 20
10 20 30
10 20 30 25
20 30 25
30 25
25
The queue is empty.
```

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Объясните понятие **абстрактного типа данных**, приведите примеры
2. Объясните понятия стека, очереди, списка и множества; какие операции поддерживают эти АТД
3. Объясните принципы реализации **односвязного** и **двусвязного** списка с использованием динамического распределения памяти
4. Какова **вычислительная сложность** операций с **односвязным** и **двусвязным** списком?
5. Объясните, как реализуются **динамические** операции на основе массива с использованием **счетчика** количества элементов
6. Объясните, как реализуются **динамические** операции на основе массива с использованием **пометки** элементов
7. Какова **вычислительная сложность** операций с **множеством** на основе упорядоченного и неупорядоченного массива?
8. Как можно оптимизировать операцию удаления первого элемента при реализации очереди на основе массива?
9. Какие встроенные реализации АТД предусмотрены в языках программирования?

ПРАКТИЧЕСКОЕ ЗАДАНИЕ 2

ПОДРОБНЕЕ – В ПРИЛОЖЕННЫХ ФАЙЛАХ

Задание 2.1. Экспериментально исследуйте вычислительную сложность реализации **списка** на базе **массива** и **двусвязного списка**. Рассмотрите **самостоятельные** реализации и **встроенные**, например ArrayList и LinkedList в Java. Схему экспериментов и программный инструментарий разработайте самостоятельно

Задание 2.2. Экспериментально исследуйте вычислительную сложность реализации добавления/удаления/поиска для **множества**. Две реализации: на основе **упорядоченного** массива и **неупорядоченного**. Исследуйте вычислительную сложность на различных сериях входных запросов, отличающихся долями этих операций в общем количестве запросов. Для экспериментов разработайте свой или адаптируйте встроенный ГПСЧ, чтобы можно было случайно, но с заданными долями выбирать вид запроса

Задание 2.3. Разработайте формат файла для **кодирования** операций вставки/удаления/поиска для множества. Например, **add5add7add10remove7contains5contains7contains10contains50** или **a5a7a10r7c5c7c10c50**. Программа должна предусматривать возможность принимать такую строку двумя способами: через **файл** и через **пользовательский ввод** (консоль/форма). Результатом работы программы должен стать вывод наподобие: 45 добавлен, 27 добавлен 27 удален 45 найден 27 не найден

Спасибо за внимание !
